

Содержание

Тест

Часть 1: Вложенные классы

Задание 1.1: Нестатический внутренний класс

Задание 1.2: Статический вложенный класс — Builder Pattern

Задание 1.3: Сравнение Inner vs Static Nested

Часть 2: Вложенные интерфейсы

Задание 2.1: Система событий

Часть 3: Обобщения (Generics)

Задание 3.1: Обобщённый стек

Задание 3.2: Ограниченные параметры типа

Задание 3.3: Wildcards — копирование коллекций

Часть 4: Исключения

Задание 4.1: Базовая обработка исключений

Задание 4.2: Собственные исключения

Задание 4.3: Try-with-resources

Задание 4.4: Цепочка исключений

Часть 5: Самостоятельная работа

Задание 5.1: Обобщённый кэш

Задание 5.2: Обобщённый Result тип

- Задание 6: Контрольные вопросы

Тест

▼ result

Ваш результат: 45 из 45

Отлично! Вы прекрасно усвоили материал.

Пройти заново

Тест 4: Вложенные классы, Обобщения и Исключения (Лекция 4)

Вопрос 1. Что выведет следующий код? [Попробуй в jshell!](#)

java

```
class Outer {  
    int x = 10;  
    class Inner {  
        int x = 20;  
        void show() {  
            System.out.println(Outer.this.x + " " + this.x);  
        }  
    }  
}  
Outer outer = new Outer();  
Outer.Inner inner = outer.new Inner();  
inner.show();
```

20 10

10 20

10 10

Ошибка компиляции

Правильно! Outer.this.x обращается к полю внешнего класса (10), а this.x — к полю внутреннего класса (20). Синтаксис Outer.this позволяет различать одноимённые члены.

Вопрос 2. Как правильно создать экземпляр нестатического внутреннего класса Inner, если он объявлен внутри класса Outer?

Inner i = new Inner();

Outer.Inner i = new Outer.Inner();

Inner i = Outer.new Inner();

Outer o = new Outer(); Outer.Inner i = o.new Inner();

Правильно! Нестатический внутренний класс привязан к экземпляру внешнего класса, поэтому сначала создаётся Outer, а затем через него — Inner: `outer.new Inner()`.

Вопрос 3. Какое ограничение имеет нестатический внутренний класс (inner class)?

Не может существовать без экземпляра внешнего класса

Не имеет доступа к `private` полям внешнего класса

Может существовать без экземпляра внешнего класса

Не может реализовывать интерфейсы

Правильно! Нестатический внутренний класс хранит неявную ссылку на экземпляр внешнего класса и не может существовать без него. Для создания объекта Inner нужен объект Outer: `outer.new Inner()`.

Вопрос 4. Как правильно создать экземпляр статического вложенного класса StaticNested, объявленного внутри класса Outer?

Outer o = new Outer(); Outer.StaticNested sn = o.new StaticNested();

StaticNested sn = new StaticNested();

Outer.StaticNested sn = new Outer.StaticNested();

Outer.StaticNested sn = Outer.new StaticNested();

Правильно! Статический вложенный класс не привязан к экземпляру внешнего класса. Он создаётся через имя внешнего класса: new Outer.StaticNested().

Вопрос 5. К каким членам внешнего класса имеет доступ статический вложенный класс?

Ко всем членам, включая private нестатические

Только к статическим членам внешнего класса

Только к public членам внешнего класса

Ни к каким — он полностью изолирован

Правильно! Статический вложенный класс не имеет ссылки на экземпляр внешнего класса, поэтому может обращаться только к статическим членам Outer, включая private static.

Вопрос 6. Что выведет следующий код? [Попробуй в jshell!](#)

Копировать

```
class Outer {  
    private static int count = 42;  
    static class Nested {  
        void printCount() {  
            System.out.println(count);  
        }  
    }  
}  
new Outer.Nested().printCount();
```

Ошибка компиляции: нет доступа к private полю

Ошибка компиляции: нельзя создать Nested без Outer

0

42

Правильно! Статический вложенный класс имеет доступ к private static полям внешнего класса. Поэтому Nested видит count = 42 и выводит его.

Вопрос 7. Почему паттерн Builder часто реализуется как статический вложенный класс?

Потому что Builder не нуждается в экземпляре внешнего класса — он сам создаёт этот экземпляр

Потому что только статический класс может иметь конструктор

Потому что статический класс автоматически реализует Serializable

Потому что нестатический класс не может возвращать объекты

Правильно! Builder создаёт экземпляр внешнего класса, а не наоборот. Он не нуждается в готовом экземпляре Outer, поэтому логично делать его статическим вложенным классом.

Вопрос 8. Что произойдёт при компиляции данного кода?

[Попробуй в jshell!](#)

```
class Outer {  
    int value = 5;  
    static class Nested {  
        void print() {  
            System.out.println(value);  
        }  
    }  
}
```

java

Выведет 5

Выведет 0

Ошибка компиляции: статический класс не может обращаться к нестатическому полю

Ошибка выполнения

Правильно! Статический вложенный класс Nested не имеет ссылки на экземпляр Outer, поэтому обращение к нестатическому полю value件不可能 — ошибка компиляции.

Вопрос 9. Какой из четырёх типов вложенных классов НЕ привязан к экземпляру внешнего класса?

Статический вложенный класс (static nested class)

Нестатический внутренний класс (inner class)

Локальный класс (local class)

Анонимный класс (anonymous class)

Правильно! Статический вложенный класс — единственный тип вложенного класса, который не привязан к экземпляру внешнего класса. Inner, local и anonymous классы требуют экземпляр.

Вопрос 10. Какой модификатор доступа может иметь интерфейс, объявленный внутри класса?

Только public

Любой: public, protected, package-private или private

Только public или private

Только public или protected

Правильно! Интерфейс, объявленный внутри класса, может иметь любой модификатор доступа: public, protected, package-private или private.

Вопрос 11. Какие неявные модификаторы получает интерфейс, объявленный внутри другого интерфейса?

private abstract

protected abstract

public static

public abstract

Правильно! Интерфейс, объявленный внутри другого интерфейса, неявно является public static. Это логично, так как все члены интерфейса по умолчанию public.

Вопрос 12. Что выведет следующий код? [Попробуй в jshell!](#)

java

```
class Container {  
    interface Printable {  
        void print();  
    }  
}  
class MyPrinter implements Container.Printable {  
    public void print() { System.out.println("Работает!"); }  
}  
new MyPrinter().print();
```

Ошибка компиляции: нельзя реализовать вложенный интерфейс

Ошибка компиляции: интерфейс Printable не виден

Ошибка выполнения

Работает!

Правильно! Вложенный интерфейс Container.Printable неявно является static и может быть реализован любым классом через implements Container.Printable.

Вопрос 13. Может ли внешний класс сам реализовать свой вложенный интерфейс?

Да, внешний класс может реализовать свой собственный вложенный интерфейс

Нет, это вызовет циклическую зависимость

Только если интерфейс объявлен как static

Только если интерфейс объявлен как public

Правильно! Да, класс может реализовать собственный вложенный интерфейс. Это не вызывает циклической зависимости и является допустимым паттерном.

Вопрос 14. Какую основную проблему решают обобщения (generics) в Java?

Ускоряют выполнение программы за счёт специализации кода

Позволяют использовать примитивные типы в коллекциях

Обеспечивают типобезопасность на этапе компиляции и устраняют необходимость явного приведения типов

Автоматически сериализуют объекты в JSON

Правильно! Generics обеспечивают типобезопасность на этапе компиляции: ошибки типов обнаруживаются до запуска, и не нужно явно приводить типы при извлечении из коллекций.

Вопрос 15. Какие общепринятые имена параметров типов используются в Java?

A — Any, B — Base, C — Class, D — Data

T — Type, E — Element, K — Key, V — Value, N — Number, R — Result

X — eXtended, Y — tYpe, Z — siZe

G — Generic, P — Parameter, S — Specific

Правильно! Общепринятые имена: T (Type), E (Element), K (Key), V (Value), N (Number), R (Result). Это конвенция, облегчающая чтение обобщённого кода.

Вопрос 16. Что выведет следующий код? [Попробуй в jshell!](#)

Копировать

```
class Box<T> {  
    private T value;  
    Box(T value) { this.value = value; }  
    T get() { return value; }  
}  
Box<String> box = new Box<>("Hello");  
System.out.println(box.get().length());
```

Ошибка компиляции

Hello

0

5

Правильно! Box хранит строку "Hello". Метод get() возвращает String, у которого вызывается length().
"Hello".length() = 5.

Вопрос 17. Что означает запись `T extends Number` в объявлении обобщённого класса?

`T` может быть только `Number` или его подклассом (`Integer`, `Double` и т.д.)

`T` должен быть точно `Number`, без подклассов

`T` может быть любым классом, который содержит число

`T` наследует `Number` и получает новые методы

Правильно! Конструкция `T extends Number` задаёт верхнюю границу: `T` может быть `Number` или любым его подклассом (`Integer`, `Double`, `BigDecimal` и т.д.).

Вопрос 18. Какая запись корректна для задания множественных границ типового параметра?

`T extends Comparable<T>, Cloneable`

`T implements Comparable<T> & Cloneable`

`T extends Comparable<T> & Cloneable`

`T super Comparable<T> & Cloneable`

Правильно! Множественные границы задаются через `&`: `T extends Comparable<T> & Cloneable`. Запятая разделяет параметры типов, а не границы одного параметра.

Вопрос 19. Как объявить обобщённый метод, принимающий массив элементов типа T и возвращающий список?

List<T> toList(T[] arr) — параметр T определён на уровне класса

<T> List<T> toList(T[] arr) — параметр T объявлен перед возвращаемым типом

List<T> <T> toList(T[] arr) — параметр T после возвращаемого типа

generic T List<T> toList(T[] arr)

Правильно! Обобщённый метод объявляет параметр типа <T> перед возвращаемым типом: <T> List<T> toList(T[] arr). Это позволяет методу работать с любым типом.

Вопрос 20. Что происходит с обобщёнными типами в процессе стирания типов (type erasure)?

Параметры типов сохраняются в байткоде для проверки во время выполнения

Типы заменяются на void

Компилятор создаёт отдельный класс для каждого параметра типа

Параметры типов существуют только при компиляции и заменяются на Object (или на границу, если указана) в байткоде

Правильно! Type erasure: компилятор проверяет типы, затем стирает параметры, заменяя их на Object (или границу). В runtime информация о типовых параметрах отсутствует.

Вопрос 21. Что выведет следующий код? [Попробуй в jshell!](#)

```
List<String> strings = new ArrayList<>();  
List<Integer> ints = new ArrayList<>();  
System.out.println(strings.getClass() == ints.getClass());
```

Копировать 

true

false

Ошибка компиляции

Ошибка выполнения

Правильно! Из-за стирания типов List<String> и List<Integer> в runtime имеют один и тот же класс ArrayList. Поэтому getClass() возвращает одинаковый объект.

Вопрос 22. Какое из следующих действий НЕВОЗМОЖНО из-за стирания типов?

Объявить переменную типа T

Привести объект к типу T

Создать экземпляр new T() или массив new T[]

Передать объект типа T как параметр метода

Правильно! Из-за стирания типов T не существует в runtime, поэтому нельзя создать new T() или new T[]. Компилятор не знает конкретный тип для создания экземпляра.

Вопрос 23. Какой класс корректно наследует обобщённый Box<T> с фиксированным типом String?

```
class Box<T> { T value; }
```

java

```
class StringBox<String> extends Box<String> {}
```

```
class StringBox extends Box<String> {}
```

```
class StringBox extends Box<T> {}
```

```
class StringBox implements Box<String> {}
```

Правильно! StringBox extends Box<String> фиксирует тип параметра. Запись StringBox<String> ошибочна: String стало бы именем нового типового параметра, затеняющего класс String.

Вопрос 24. Что означает подстановочный знак ? (wildcard) в обобщениях?

Означает тип Object

Означает отсутствие типа

Означает ошибку в типе

Означает неизвестный тип — может быть любым

Правильно! Wildcard ? означает «неизвестный тип». Используется когда конкретный тип не важен, например List<?> принимает список с элементами любого типа.

Вопрос 25. Какой параметр метода позволит принять List<Integer>, List<Double> и List<Number>?

List<? extends Number>

List<? super Number>

List<Number>

List<Object>

Правильно! List<? extends Number> принимает List любого подтипа Number: List<Integer>, List<Double>, List<Number>. List<Number> не примет List<Integer> из-за инвариантности.

Вопрос 26. Что означает правило PECS (Producer Extends, Consumer Super)?

Продюсер должен наследовать Consumer, а Consumer — расширять Producer

extends используется для записи, super — для чтения

Если из коллекции только читаем (producer) — используем ? extends T; если только записываем (consumer) — используем ? super T

Producer и Consumer — стандартные интерфейсы Java

Правильно! PECS: если коллекция — источник данных (Producer), используем extends для чтения; если коллекция — приёмник данных (Consumer), используем super для записи.

Вопрос 27. Почему в сигнатуре `Collections.copy(List<? super T> dest, List<? extends T> src)` используются именно такие wildcards?

Это просто конвенция, можно использовать и наоборот

`src` — производитель данных (читаем элементы, `extends`), `dest` — потребитель данных (записываем элементы, `super`)

`super` всегда используется для источника, `extends` — для назначения

Оба параметра можно заменить на `List<T>` без потери функциональности

Правильно! В `Collections.copy()` из `src` читаем элементы (`producer` → `extends`), в `dest` записываем элементы (`consumer` → `super`). Это классическое применение PECS.

Вопрос 28. Что выведет следующий код? [Попробуй в jshell!](#)

```
class Pair<K, V> {  
    K key; V value;  
    Pair(K key, V value) { this.key = key; this.value = value; }  
}  
Pair<String, Integer> p = new Pair<>("age", 25);  
System.out.println(p.key + "=" + p.value);
```

Копировать

Ошибка компиляции

age25

null=null

age=25

Правильно! `Pair` создаётся с ключом "age" (String) и значением 25 (Integer). Конкатенация через `+` выводит `age=25`.

Вопрос 29. Какое утверждение верно относительно обобщённых интерфейсов?

```
interface Transformer<T, R> {  
    R transform(T input);  
}
```

java

Класс может реализовать интерфейс с конкретными типами: `class StringToInt implements Transformer<String, Integer>`

Интерфейс не может иметь более одного параметра типа

Реализующий класс обязан быть тоже обобщённым

Обобщённые интерфейсы нельзя использовать с анонимными классами

Правильно! Обобщённый интерфейс можно реализовать с конкретными типами. Класс не обязан быть обобщённым — он может зафиксировать типы при `implements`.

Вопрос 30. Какая иерархия исключений верна в Java?

`Object` → `Exception` → `Throwable` → `Error`

`Throwable` → `Error` и `Exception`; `Exception` → `RuntimeException`

`Exception` → `Throwable` → `Error` → `RuntimeException`

`Throwable` → `RuntimeException` → `Exception` → `Error`

Правильно! `Throwable` — корень иерархии. От него наследуются `Error` (серьёзные ошибки JVM) и `Exception` (обрабатываемые исключения). `RuntimeException` — подкласс `Exception`.

Вопрос 31. Какое исключение является проверяемым (checked)?

NullPointerException

ArrayIndexOutOfBoundsException

IOException

StackOverflowError

Правильно! IOException — checked-исключение (наследует Exception, но не RuntimeException).
NullPointerException и ArrayIndexOutOfBoundsException — unchecked (RuntimeException).

Вопрос 32. Чем checked-исключения отличаются от unchecked?

Checked-исключения должны быть обработаны (try-catch) или объявлены (throws) — иначе код не скомпилируется

Unchecked-исключения нельзя ловить в catch-блоке

Checked-исключения возникают только во время выполнения

Unchecked-исключения наследуют класс Exception напрямую

Правильно! Checked-исключения обязательно должны быть обработаны в try-catch или объявлены в throws.
Unchecked (наследники RuntimeException) не требуют этого.

Вопрос 33. К какой категории относится StackOverflowError?

Checked exception

Unchecked exception (RuntimeException)

Обычная ошибка компиляции

Error — серьёзная ошибка JVM, которую обычно не следует обрабатывать

Правильно! StackOverflowError наследует Error → Throwable. Это не исключение, а ошибка JVM (переполнение стека вызовов), которую обычно нет смысла перехватывать.

Вопрос 34. Что выведет следующий код? [Попробуй в jshell!](#)

java

```
try {  
    System.out.print("A");  
    int x = 1 / 0;  
    System.out.print("B");  
} catch (ArithmeticException e) {  
    System.out.print("C");  
} finally {  
    System.out.print("D");  
}
```

ABCD

ACD

AD

ACD, но D не выполнится если catch бросит исключение

Правильно! Выводится "A", затем деление на ноль вызывает ArithmeticException (B пропускается), catch печатает "C", и finally всегда выполняется — печатает "D". Итого: ACD.

Вопрос 35. Почему порядок catch-блоков имеет значение?

Потому что выполняются все подходящие catch-блоки последовательно

Порядок не имеет значения — JVM сама выбирает наиболее подходящий

Более конкретные исключения должны стоять раньше, иначе ошибка компиляции: суперкласс перехватит всё до подклассов

Первый catch всегда обрабатывает все исключения

Правильно! Catch-блоки проверяются по порядку. Если суперкласс стоит раньше подкласса, подкласс никогда не будет достигнут — компилятор выдаст ошибку "unreachable catch block".

Вопрос 36. Что выведет следующий код? [Попробуй в jshell!](#)

[Копировать](#)

```
try {  
    System.out.print("1");  
    throw new RuntimeException();  
} catch (RuntimeException e) {  
    System.out.print("2");  
} finally {  
    System.out.print("3");  
}  
System.out.print("4");
```

1234

123

12

134

Правильно! Выводится "1", выбрасывается RuntimeException, catch ловит его и печатает "2", finally печатает "3", затем выполнение продолжается — печатает "4". Итого: 1234.

Вопрос 37. Что позволяет конструкция multi-catch, появившаяся в Java 7?

java

```
catch (IOException | SQLException e) { ... }
```

Ловить исключения, которые являются наследниками друг друга

Обрабатывать каждое исключение отдельным блоком кода

Ловить только unchecked-исключения

Ловить несколько несвязанных типов исключений в одном catch-блоке с общей обработкой

Правильно! Multi-catch позволяет одним catch-блоком обработать несколько несвязанных типов исключений. Типы разделяются символом |, и они не должны быть наследниками друг друга.

Вопрос 38. Какой метод класса Throwable возвращает текстовое описание исключения?

toString()

printStackTrace()

getMessage()

getDescription()

Правильно! getMessage() возвращает строку-сообщение, переданную в конструктор исключения. printStackTrace() печатает стек вызовов, toString() — класс + сообщение.

Вопрос 39. В чём разница между ключевыми словами throw и throws?

throw объявляет исключения метода, throws бросает исключение

throw бросает конкретное исключение, throws объявляет в сигнатуре метода, какие checked-исключения он может выбросить

throw используется в catch, throws — в try

Разницы нет, это синонимы

Правильно! throw new Exception("msg") бросает конкретный объект исключения. throws Exception в сигнатуре метода предупреждает вызывающий код о возможном checked-исключении.

Вопрос 40. Что выведет следующий код? Попробуй в jshell!

```
try {  
    throw new Exception("Тест");  
} catch (Exception e) {  
    System.out.println(e.getMessage());  
}
```

java

Тест

Exception: Тест

java.lang.Exception: Тест

null

Правильно! getMessage() возвращает строку, переданную в конструктор: "Тест". Метод toString() вернул бы "java.lang.Exception: Тест", а getMessage() — только само сообщение.

Вопрос 41. Как правильно создать собственное checked-исключение?

class MyException extends RuntimeException {}

class MyException extends Error {}

class MyException implements Exception {}

class MyException extends Exception {}

Правильно! Для создания checked-исключения наследуем от Exception. RuntimeException — для unchecked, Error — для ошибок JVM. Exception не интерфейс, а класс.

Вопрос 42. Что такое try-with-resources и какой интерфейс должен реализовать ресурс?

Конструкция для автоматического открытия ресурсов; интерфейс Openable

Конструкция для автоматического закрытия ресурсов; интерфейс AutoCloseable

Конструкция для кеширования ресурсов; интерфейс Cacheable

Конструкция для логирования ресурсов; интерфейс Loggable

Правильно! Try-with-resources (Java 7+) автоматически закрывает ресурсы после блока try. Ресурс должен реализовать интерфейс AutoCloseable с методом close().

Вопрос 43. Что выведет следующий код? [Попробуй в jshell!](#)

```
class MyRes implements AutoCloseable {  
    public void close() { System.out.print("closed "); }  
}  
try (MyRes r = new MyRes()) {  
    System.out.print("used ");  
}
```

java

used closed

closed used

used

Ошибка компиляции

Правильно! Сначала выполняется тело try — "used ", затем ресурс автоматически закрывается — вызывается close(). который печатает "closed ". Итого: used closed.

Вопрос 44. Что такое цепочка исключений (exception chaining)?

Перехват нескольких исключений подряд в отдельных try-catch блоках

Вызов нескольких throw подряд в одном методе

Использование multi-catch для нескольких типов

Оборачивание исходного исключения (cause) в новое через throw new HighLevel("msg", cause)

Правильно! Exception chaining — оборачивание исходного исключения (cause) в новое: throw new HighLevel("msg", originalException). Это сохраняет информацию о первопричине.

Вопрос 45. Какая практика обработки исключений считается ПЛОХОЙ?

```
// Вариант А
try { riskyMethod(); }
catch (Exception e) { /* пусто */ }

// Вариант В
try { riskyMethod(); }
catch (SpecificException e) { log.error("Ошибка", e); throw e; }
```

java

Вариант В — нельзя бросать перехваченное исключение повторно

Вариант А — перехват слишком широкого Exception и «проглатывание» исключения без обработки

Оба варианта являются плохой практикой

Оба варианта являются хорошей практикой

Правильно! Вариант А — двойная ошибка: перехват слишком широкого Exception маскирует конкретные проблемы, а пустой catch «проглатывает» исключение без логирования и обработки.

Часть 1: Вложенные классы

Задание 1.1: Нестатический внутренний класс

Реализуйте класс Library с полями name и capacity. Внутри него объявите нестатический внутренний класс Book с полями title (String), author (String), year (int). Метод getInfo() должен выводить: «Книга: [title] автора [author] ([year]) в библиотеке [Library.name]». Обратите внимание: Book обращается к полю name внешнего класса напрямую. Создайте экземпляр Library, затем экземпляр Book через lib.new Book(...), вызовите getInfo().

Ожидаемый вывод:

```
Книга: Война и мир автора Толстой Л.Н. (1869) в библиотеке Городская библиотека
```

Вопрос: Попробуйте добавить static к классу Book. Что изменится? Почему потеряется доступ к name?

Решение

```
package practice_4.task_1_1;

public class Library {
    private String name;
    private int capacity;
    Library(String name, int capacity){
        this.name=name;
        this.capacity=capacity;
    }
    class Book{
        String title;
        String author;
        int year;
        Book(String title, String author, int year){
            this.title=title;
            this.author=author;
            this.year=year;
        }
        public void getInfo(){
            System.out.println("Книга: " + title + " автора " + author + " (" + year + ") " + "в библиотек"
        }
    }
    public static void main(String[] args){
        Library library = new Library("Городская библиотека", 100);
        Book book1 = library.new Book("Война и мир", "Толстой Л.Н.", 1869);
        book1.getInfo();
    }
}
```

▼ result

Книга: Война и мир автора Толстой Л.Н. (1869) в библиотеке Городская библиотека

Process finished with exit code 0

Ответ на вопрос:

- Если сделаем вложенный класс статическим, то сможем обращаться к внешнему классу только к статическим полям и методам. Нельзя создать его экземпляр через внешний экземпляр + new конструктор книги.
- Можно сделать поля во внешнем классе статическими и внутри класса статическими, убрать конструкторы, для полей добавить значения, которые используем.
- При создании экземпляры книги обращаемся к конструктору через внешний класс.

▼ result

```
package practice_4.task_1_1;

public class Library {
    2 usages
    private String name;
    1 usage
    private int capacity;
    1 usage
    Library(String name, int capacity){
        this.name=name;
        this.capacity=capacity;
    }
    2 usages
    static class Book{
        2 usages
        String title;
        2 usages
        String author;
        2 usages
        int year;
        1 usage
        Book(String title, String author, int year){
            this.title=title;
            this.author=author;
            this.year=year;
        }
        1 usage
        public void getInfo(){
            System.out.println("Книга: " + title + " автора " + author + " (" + year + ") " + "в библиотеке " + name);
        }
    }
    public static void main(String[] args){
        Library library = new Library(name: "Городская библиотека", capacity: 100);
        Book book1 = library.new Book(title: "Война и мир", author: "Толстой Л.Н.", year: 1869);
        book1.getInfo();
    }
}
```

```
C:\Users\vinna\IdeaProjects\missa\src\practice_4\task_1_1\Library.java:20:113
java: non-static variable name cannot be referenced from a static context
C:\Users\vinna\IdeaProjects\missa\src\practice_4\task_1_1\Library.java:25:22
java: qualified new of static class
```

```
package practice_4.task_1_1;

public class Library {
    1 usage
    private static String name = "Городская библиотека";
    no usages
    private static int capacity = 100;
    // Library(String name, int capacity){
    //     this.name=name;
    //     this.capacity=capacity;
    // }
    2 usages
    static class Book{
        1 usage
        static String title = "Война и мир";
        1 usage
        static String author = "Толстой Л.Н.";
        1 usage
        static int year = 1869;
        // Book(String title, String author, int year){
        //     this.title=title;
        //     this.author=author;
        //     this.year=year;
        // }
        1 usage
        public void getInfo(){
            System.out.println("Книга: " + title + " автора " + author + " (" + year + ") " + "в библиотеке " + name);
        }
    }
    public static void main(String[] args){
    //     Library library = new Library("Городская библиотека", 100);
    //     Book book1 = library.new Book("Война и мир", "Толстой Л.Н.", 1869);
    Library library = new Library();
    Library.Book book1 = new Library.Book();
    book1.getInfo();
    }
}
```

```
Книга: Война и мир автора Толстой Л.Н. (1869) в библиотеке Городская библиотека
```

```
Process finished with exit code 0
```

Задание 1.2: Статический вложенный класс — Builder Pattern

Реализуйте класс Computer с final-полями: cpu (String), ram (int), storage (int, по умолчанию 256), gpu (String, по умолчанию null), ssd (boolean, по умолчанию true). Конструктор private — принимает Builder.

Реализуйте статический вложенный класс Builder с конструктором Builder(String cpu, int ram) и методами storage(int), gpu(String), ssd(boolean) (каждый возвращает this) и build() (возвращает новый Computer).

Переопределите toString() в формате: Computer{CPU='...', RAM=...GB, Storage=...GB SSD/HDD, GPU='...' / 'встроенная'}.

Создайте игровой компьютер (Core i9, 32GB, 2000GB, RTX 4080, SSD) и офисный (Core i5, 16GB, дефолтные значения) и выведите оба.

Решение

```
package practice_4.task_1_2;

public class Computer {
    private final String cpu;
    private final int ram;
    private final int storage;
    private final String gpu;
    private final boolean ssd;
    private Computer(Builder builder){
        this.cpu= builder.cpu;
        this.ram= builder.ram;
        this.storage= builder.storage;
        this.gpu= builder.gpu;
        this.ssd= builder.ssd;
    }
    static class Builder{
        private final String cpu;
        private final int ram;
        private int storage = 256;
        private String gpu = null;
        private boolean ssd = true;
        Builder (String cpu, int ram){
            this.cpu=cpu;
            this.ram=ram;
        }
        Builder storage(int storage){
            this.storage=storage;
            return this;
        }
        Builder gpu(String gpu){
            this.gpu=gpu;
            return this;
        }
        Builder ssd(boolean ssd){
            this.ssd=ssd;
            return this;
        }
        Computer build(){return new Computer(this);}
    }
    @Override
    public String toString() {
        String storageType = ssd ? "SSD":"HDD";
        String gpuType = (gpu == null || gpu.isEmpty()) ? "встроенная" : "'" + gpu + "'";
        return "Computer{" +
            "cpu='" + cpu + '\'' +
            ", ram=" + ram + "GB" +
```

```

        ", storage=" + storage + "GB" + storageType +
        ", gpu='" + gpuType + '\\'' + '}'';
    }
    public static void main(String[] args){
        Computer computerPlay = new Builder("Core i9", 32).storage(2000).gpu("RTX 4080").ssd(true).build();
        System.out.println(computerPlay);
        Computer computerOffice = new Builder("Core i5", 16).build();
        System.out.println(computerOffice);
    }
}

```

▼ result

```

Computer{cpu='Core i9', ram=32GB, storage=2000GBSSD, gpu='RTX 4080'}
Computer{cpu='Core i5', ram=16GB, storage=256GBSSD, gpu='встроенная'}

Process finished with exit code 0

```

Задание 1.3: Сравнение Inner vs Static Nested

Запустите следующий код и объясните результаты:

```

public class Memory {
    private int instanceData = 100;
    private static int staticData = 200;

    class InnerClass {
        void show() {
            System.out.println("Inner: instanceData = " + instanceData); // ?
            System.out.println("Inner: staticData = " + staticData);      // ?
        }
    }

    static class StaticNested {
        void show() {
            // System.out.println("Static: instanceData = " + instanceData); // ?
            System.out.println("Static: staticData = " + staticData);      // ?
        }
    }

    public static void main(String[] args) {
        Memory memory = new Memory();

        // Нестатический – нужен экземпляр
        InnerClass inner = memory.new InnerClass();
        inner.show();

        // Статический – экземпляр не нужен
        StaticNested nested = new StaticNested();
    }
}

```

```
        nested.show();
    }
}
```

Демонстрация без закомментированной строки и с убиранием комментирования.

▼ result

```
Inner: instanceData = 100
Inner: staticData = 200
Static: staticData = 200

Process finished with exit code 0
```

```
C:\Users\vinna\IdeaProjects\misa\src
\practice_4\task_1_3\Memory.java:16:61
java: non-static variable instanceData
cannot be referenced from a static
context
```

Ответьте письменно:

1. Почему закомментированная строка вызовет ошибку компиляции?
 - Нельзя обращаться к нестатическим переменным внешнего класса из вложенного статического класса (статический вложенный класс не имеет ссылки на экземпляр внешнего класса).
2. В чём разница при создании экземпляров этих двух классов?
 - При создании нестатического экземпляра всегда делаем через объект внешнего класса.
 - При создании статического экземпляра, создаём через статический вложенный класс, не привязан к внешнему классу.

Часть 2: Вложенные интерфейсы

Задание 2.1: Система событий

Реализуйте класс EventSystem со следующими вложенными элементами:

- Вложенный интерфейс EventHandler с методом void handle(T event).
- Вложенный интерфейс EventFilter с методом boolean accept(T event).
- Вложенный статический класс Event с полями type (String), data (String), timestamp (long, устанавливается при создании).
- В самом EventSystem: список обработчиков (List<EventHandler>), текущий фильтр (EventFilter), методы setFilter, addHandler, fire.

В методе fire(Event event): если фильтр есть и он не принимает событие — вывести «Событие отфильтровано: ...»; иначе — передать всем обработчикам.

В main(): установите фильтр, пропускающий только type == "ERROR". Добавьте обработчик, выводящий событие. Запустите 4 события (INFO, ERROR, DEBUG, ERROR).

Решение

```
package practice_4.task_2_1;

import java.util.ArrayList;
import java.util.List;

public class EventSystem {
    private List<EventHandler<Event>> handlers = new ArrayList<>();
    private EventFilter<Event> filter;
    public interface EventHandler<T>{
        void handle(T event);
    }
    public interface EventFilter<T>{
        boolean accept(T event);
    }
    static class Event{
        private String type;
        private String data;
        private long timestamp;
        public Event(String type, String data){
            this.type=type;
            this.data=data;
            this.timestamp=System.currentTimeMillis();
        }
        public String getType() {return type;}
        public String getData() {return data;}
        public long getTimestamp() {return timestamp;}

        @Override
        public String toString() {return "Event{" + "type='" + type + '\'' + ", data='" + data + '\'' +
    }

    public void addHandlers(EventHandler<Event> handler) {handlers.add(handler);}
    public void setFilter(EventFilter<Event> filter) {this.filter = filter;}
    public void fire(Event event){
        if (event != null || !filter.accept(event)) System.out.println("Событие отфильтровано: " + event)
        else{
            for (EventHandler<Event> handler : handlers){
                handler.handle(event);
            }
        }
    }
}
```



```

    }
}
}
public static void main(String[] args){
    EventSystem system = new EventSystem();
    system.setFilter(event -> "ERROR".equals(event.getType()));
    system.addHandlers(event -> System.out.println("Обработано: " + event));
    system.fire(new Event("INFO", "Запуск сервера"));
    system.fire(new Event("ERROR", "Ошибка подключения к бд"));
    system.fire(new Event("DEBUG", "Входим в API, в эндпоинт"));
    system.fire(new Event("ERROR", "Освободи памяти"));
}
}

```

▼ result

```

Событие отфильтровано: Event{type='INFO', data='Запуск сервера', timestamp=1776971132670}
Событие отфильтровано: Event{type='ERROR', data='Ошибка подключения к бд', timestamp=1776971132693}
Событие отфильтровано: Event{type='DEBUG', data='Входим в API, в эндпоинт', timestamp=1776971132693}
Событие отфильтровано: Event{type='ERROR', data='Освободи памяти', timestamp=1776971132693}

Process finished with exit code 0

```

Часть 3: Обобщения (Generics)

Задание 3.1: Обобщённый стек

Реализуйте обобщённый стек Stack (LIFO) на основе массива Object[]:

- Начальная ёмкость 10; при переполнении массив расширяется вдвое.
- push(T element) — добавляет на вершину.
- pop() — удаляет и возвращает вершину; бросает EmptyStackException если пуст.
- peek() — возвращает вершину без удаления; бросает EmptyStackException если пуст.
- isEmpty(), size(), toString().

Продемонстрируйте работу: создайте Stack, добавьте три строки, вызовите peek() и pop(). Создайте Stack, заполните числами 10–50, извлеките все элементы в цикле.

Решение

```

package practice_4.task_3_1;

import java.util.Arrays;

```

```

import java.util.EmptyStackException;

public class Stack<T>{
    private Object[] objects;
    private int size = 0;
    private static final int capacity = 10;
    public Stack(){
        this.objects = new Object[capacity];
    }
    public void push(T element){
        if (size == objects.length){
            resize();
        }
        objects[size++]=element;
    }
    public T pop(){
        if (isEmpty()){
            throw new EmptyStackException();
        }
        T element = (T) objects[--size];
        objects[size] = null;
        return element;
    }
    public T peek(){
        if (isEmpty()){
            throw new EmptyStackException();
        }
        T element = (T) objects[size-1];
        return element;
    }
    public boolean isEmpty(){return size == 0;}
    public int size() {return size;}
    public void resize(){
        int newCapacity = objects.length * 2;
        objects = Arrays.copyOf(objects, newCapacity);
    }
    @Override
    public String toString() {
        return "Stack{" +
            "objects=" + Arrays.toString(objects) +
            '}';
    }
}

class Test{
    public static void main(String[] args){
        Stack<String> stack1 = new Stack<>();
        stack1.push("hello");
        stack1.push("world");
        stack1.push("bye");
        System.out.println(stack1.peek());
        System.out.println(stack1.pop());

        Stack<Integer> stack2 = new Stack<>();
        for (int i=10;i<=50;i++) {
            stack2.push(i);
        }
    }
}

```

```

    }
    while(!stack2.isEmpty()){
        System.out.print(stack2.pop() + " ");
    }
}
}

```

▼ result

```

bye
bye
50 49 48 47 46 45 44 43 42 41 40 39 38 37 36 35 34 33 32 31 30 29 28 27 26 25 24 23 22 21 20 19 18 17 16 15 14 13 12 11 10
Process finished with exit code 0

```

Задание 3.2: Ограниченные параметры типа

Реализуйте утилитарный класс `NumberUtils` со следующими статическими методами:

- `sum(List<? extends Number> numbers) → double` — сумма элементов.
- `average(List<? extends Number> numbers) → double` — среднее; бросает `IllegalArgumentException` для пустого списка.
- `<T extends Comparable> T findMax(List list)` — максимальный элемент; бросает `IllegalArgumentException` для `null` или пустого списка.
- `<T extends Comparable> T findMin(List list)` — аналогично.
- `fillWithIntegers(List<? super Integer> list, int n)` — добавляет в список числа от 1 до `n`.

Протестируйте: сумма `[1,2,3,4,5]`, сумма `[1.5,2.5,3.5]`, среднее, максимум строк `["яблоко","апельсин","банан"]`, `fillWithIntegers` в `List`.

Решение

```

package practice_4.task_3_2;

import java.util.ArrayList;
import java.util.Arrays;
import java.util.List;

public class NumberUtils<T> {
    public static double sum(List<? extends Number> numbers){
        double sum = 0;
        for (Number number : numbers){
            sum = sum + number.doubleValue();
        }
        return sum;
    }

    public static double average(List<? extends Number> numbers){
        if (numbers == null || numbers.isEmpty()){
            throw new IllegalArgumentException("Пустой список");
        }
    }
}

```

```

        return sum(numbers) / numbers.size();
    }
    public static <T extends Comparable<T>> T findMax(List<T> list){
        if (list == null || list.isEmpty()){
            throw new IllegalArgumentException("Пустой список");
        }
        T max = (T) list.get(0);
        for (int i=1;i<list.size();i++){
            if (list.get(i).compareTo(max) > 0){
                max = (T) list.get(i);
            }
        }
        return max;
    }
    public static <T extends Comparable<T>> T findMin(List<T> list){
        if (list == null || list.isEmpty()){
            throw new IllegalArgumentException();
        }
        T min = (T) list.get(0);
        for (int i=1;i<list.size();i++){
            if (list.get(i).compareTo(min) < 0){
                min = (T) list.get(i);
            }
        }
        return min;
    }
    public static void fillWithIntegers(List<? super Integer> list, int n){
        for (int i=1;i<=n;i++){
            list.add(i);
        }
    }
}
class TestNumberUtils{
    public static void main(String[] args){
        List<Integer> intList = Arrays.asList(1, 2, 3, 4, 5);
        double sumInt = NumberUtils.sum(intList);
        System.out.println("Сумма [1, 2, 3, 4, 5]: " + sumInt);

        List<Double> doubleList = Arrays.asList(1.5, 2.5, 3.5);
        double sumDouble = NumberUtils.sum(doubleList);
        System.out.println("Сумма [1.5, 2.5, 3.5]: " + sumDouble);

        double avgInt = NumberUtils.average(intList);
        double avgDouble = NumberUtils.average(doubleList);
        System.out.println("Среднее для intList: " + avgInt);
        System.out.println("Среднее для doubleList: " + avgDouble);

        List<String> stringList = Arrays.asList("яблоко", "апельсин", "банан");
        String maxString = NumberUtils.findMax(stringList);
        System.out.println("Максимум из строк: " + maxString);

        List<Number> numberList = new ArrayList<>();
        int n = 5;
        NumberUtils.fillWithIntegers(numberList, n);
        System.out.println("Итоговый заполненный List<Number>: " + numberList); // Ожидается: [1, 2, 3,
    }
}

```

```
}
```

▼ result

```
Сумма [1, 2, 3, 4, 5]: 15.0
Сумма [1.5, 2.5, 3.5]: 7.5
Среднее для intList: 3.0
Среднее для doubleList: 2.5
Максимум из строк: яблоко
Итоговый заполненный List<Number>: [1, 2, 3, 4, 5]

Process finished with exit code 0
```

Задание 3.3: Wildcards — копирование коллекций

Реализуйте класс WildcardDemo с тремя статическими методами, правильно выбрав wildcards:

- `printAll(List<?> list)` — выводит все элементы (принимает список любого типа).
- `sumNumbers(List<? extends Number> list) → double` — сумма через `Number::doubleValue`.
- `addDefaults(List<? super String> list)` — добавляет "default1" и "default2".

Объясните в комментариях: почему `List` для `Integer` не присваивается переменной `List` для `Number`? Как это обойти с помощью wildcard? Продемонстрируйте корректную копию из `List` для `Integer` в `List` для `Number`.

Решение

```
package practice_4.task_3_3;

import java.util.*;

public class WildcardDemo {
    public static void printAll(List<?> list){
        for (int i=0;i<list.size();i++){
            System.out.print(list.get(i));
        }
    }

    public static double sumNumbers(List<? extends Number> list){
        double sum = list.stream().mapToDouble(Number::doubleValue).sum();
        return sum;
    }

    public static void addDefaults(List<? super String> list){
        list.add("default1");
        list.add("default2");
    }
}
```

```

class Test{
    public static void main(String[] args){
        List<Integer> list1 = Arrays.asList(1, 2, 3, 4, 5);
        List<Number> list2 = new ArrayList<>();
        copyElements(list1, list2);
        System.out.println(list2);
    }
}

```

- В Java массивы являются ковариантными (например, Integer[] можно присвоить в Number[] = прозрачная коробка яблок разрешит набить табличку фрукты, но положить апельсины нельзя = яблоки инты, апельсины вещественные).
- Дженирики = инвариантны:
- List для Integer и List для Number — это два абсолютно разных, не связанных друг с другом типа, у которых общий предок только List<?> (закрытая сумка только для яблок с такой надписью, сразу говорит, что и для чего).

Правило PESC

- Источник — это коллекция, которая отдает (производит) данные. Мы из нее только читаем.
- Приемник — это коллекция, которая принимает (потребляет) данные. Мы в нее только записываем.
- Используем extends, когда цель — безопасное чтение из коллекции. Нам важен общий верхний тип, к которому можно привести всё содержимое.
- Используем super, когда цель — безопасная запись в коллекцию. Нам важно, чтобы коллекция могла принять наш конкретный тип.

Пример: Animal <- Dog <- Cat.

1. Есть метод, принимающий список собак, заставляя их всей лаять и пересчитывать их (источник).

```

public static void processDogs(List<? extends Dog> source) {
    for (Dog dog : source) {
        dog.bark(); // БЕЗОПАСНО: Кем бы ни были элементы, они как минимум Dog!
    }
    //source.add(new Dog()); // ОШИБКА компиляции! Почему?
}

```

Чтение разрешено:

- Метод может принять List для Dog или List для Corgi. Если передали List для Corgi, то каждый Корги — это Собака, поэтому вызвать метод bark() абсолютно безопасно.

Запись запрещена:

- Если бы компилятор разрешил строку source.add(new Dog()), а нам на вход передали чистый список List, то мы бы только что подселили обычную Собаку в вольер к Корги. Это сломало бы

логику программы, где ожидаются только Корги.

2. Теперь мы пишем метод, который берет конкретную собаку `class Corgi` и добавляет ее в список-приемник (приёмник).

```
public static void addDogToKennel(List<? super Dog> destination) {  
    destination.add(new Dog()); // БЕЗОПАСНО: Обычную собаку можно добавить  
    destination.add(new Corgi()); // БЕЗОПАСНО: Корги тоже является собакой!  
  
    // Dog dog = destination.get(0); // ОШИБКА компиляции! Почему?  
}
```

Запись разрешена:

- Метод может принять `List` для `Dog`, `List` для `Animal` или `List` для `Object`.
- В любой из этих трех списков можно абсолютно легально положить объект типа `Dog` или его наследника `Corgi` (ведь Корги — это тоже Собака и тоже Животное).

Чтение запрещено:

- Если нам на вход передали `List` для `Animal`, то там внутри уже могут лежать кошки, птицы и крокодилы.
- Попытка прочитать оттуда элемент и сразу привести его к типу `Dog` (строка `Dog dog = destination.get(0)`) приведет к катастрофе в runtime, если там окажется кошка.
- Единственное, что гарантированно можно оттуда прочитать — это `Object`.

▼ result

```
[1, 2, 3, 4, 5]
```

```
Process finished with exit code 0
```

Часть 4: Исключения

Задание 4.1: Базовая обработка исключений

Напишите программу `ExceptionBasics` с тремя блоками:

- Деление: для массива делителей `{2, 0, 5, 0, 1}` вычислите `100 / divisor`, обработав `ArithmeticException`.

- Массив: для индексов {0, 1, 5, 2, -1} обратитесь к массиву {10, 20, 30}, обработав `ArrayIndexOutOfBoundsException`.
- Парсинг: для строк {"42", "abc", "100", "3.14", "-7"} вызовите `Integer.parseInt`, обработав `NumberFormatException`.

Для каждого случая распечатайте результат или сообщение об ошибке.

Решение

```
package practice_4.task_4_1;

public class ExceptionBasic {
    public static void main(String[] args){
        int[] arr1 = {2, 0, 5, 0, 1};
        for (int i=0;i<arr1.length;i++) {
            try {
                int divisor = arr1[i];
                System.out.println(100 / divisor);
            } catch (ArithmeticException e) {
                System.out.println("Делить на 0 нельзя" + e);
            }
        }

        int[] arr2 = {10,20,30};
        int[] arr2index = {0, 1, 5, 2, -1};
        for (int i=0;i<arr2index.length;i++) {
            try {
                System.out.println(arr2[arr2index[i]]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Выход за границы массива");
            }
        }

        String[] arr3 = {"42", "abc", "100", "3.14", "-7"};
        for (String s : arr3) {
            try {
                int temp = Integer.parseInt(s);
                System.out.println(temp);
            } catch (NumberFormatException e) {
                System.out.println("Не числа");
            }
        }
    }
}
```

▼ result


```
50
Делить на 0 нельзяjava.lang.ArithmeticException: / by zero
20
Делить на 0 нельзяjava.lang.ArithmeticException: / by zero
100
10
20
Выход за границы массива
30
Выход за границы массива
42
Не числа
100
Не числа
-7

Process finished with exit code 0
```

Задание 4.2: Собственные исключения

Создайте систему исключений для банковского приложения:

Решение

```
package practice_4.task_4_2;

public class BankException extends Exception{
    private String operationId;

    public BankException(String message, String operationId) {
        super(message);
        this.operationId = operationId;
    }

    public String getOperationId() { return operationId; }
}

package practice_4.task_4_2;

public class InsufficientFundsException extends BankException{
    private double required;
    private double available;

    public InsufficientFundsException(double required, double available, String operationId) {
```

```

        super(String.format("Недостаточно средств. Требуется: %.2f, доступно: %.2f",
            required, available), operationId);
        this.required = required;
        this.available = available;
    }

    public double getRequired() { return required; }
    public double getAvailable() { return available; }
}

```

```
package practice_4.task_4_2;
```

```

public class InvalidAmountException extends RuntimeException{
    public InvalidAmountException(double amount) {
        super("Сумма должна быть положительной, получено: " + amount);
    }
}

```

```
package practice_4.task_4_2;
```

```

public class BankAccount {
    private String accountId;
    private double balance;

    public BankAccount(String accountId, double initialBalance) {
        if (initialBalance < 0) {
            throw new InvalidAmountException(initialBalance);
        }
        this.accountId = accountId;
        this.balance = initialBalance;
    }

    public void deposit(double amount) {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        balance += amount;
        System.out.printf("Пополнение на %.2f. Баланс: %.2f\n", amount, balance);
    }

    public void withdraw(double amount) throws InsufficientFundsException {
        if (amount <= 0) {
            throw new InvalidAmountException(amount);
        }
        if (amount > balance) {
            throw new InsufficientFundsException(amount, balance, "WD-" + accountId);
        }
        balance -= amount;
        System.out.printf("Снятие %.2f. Баланс: %.2f\n", amount, balance);
    }

    public double getBalance() { return balance; }
}

```

```

    public String getAccountId() { return accountId; }
}

```

```

package practice_4.task_4_2;

```

```

public class BankTest {
    public static void main(String[] args){
        BankAccount account = new BankAccount("ACC-001", 1000.0);

        // Тест нормального пополнения и снятия
        account.deposit(500.0);
        try {
            account.withdraw(200.0);
        } catch (InsufficientFundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        // Тест недостаточных средств
        try {
            account.withdraw(2000.0); // Должно бросить InsufficientFundsException
        } catch (InsufficientFundsException e) {
            System.out.println("Ошибка: " + e.getMessage());
            System.out.printf("Не хватает: %.2f%n", e.getRequired() - e.getAvailable());
        }

        // Тест невалидной суммы (unchecked)
        try {
            account.deposit(-100); // Должно бросить InvalidAmountException
        } catch (InvalidAmountException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        // Тест невалидного начального баланса
        try {
            BankAccount badAccount = new BankAccount("BAD-001", -500);
        } catch (InvalidAmountException e) {
            System.out.println("Нельзя создать счёт: " + e.getMessage());
        }
    }
}

```

▼ result

```
Пополнение на 500,00. Баланс: 1500,00
Снятие 200,00. Баланс: 1300,00
Ошибка: Недостаточно средств. Требуется: 2000,00, доступно: 1300,00
Не хватает: 700,00
Ошибка: Сумма должна быть положительной, получено: -100.0
Нельзя создать счёт: Сумма должна быть положительной, получено: -500.0

Process finished with exit code 0
```

Задание 4.3: Try-with-resources

- Изучите класс `FileLogger`, реализующий `AutoCloseable`.
- Запустите программу и убедитесь, что `close()` вызывается автоматически.

Объясните:

- (1) что выводится при `close()`?
- (2) почему `transient` не применяется здесь, а `flush()` нужен?

Решение

```
package practice_4.task_4_3;

import java.io.*;
import java.time.LocalDateTime;
import java.time.format.DateTimeFormatter;

public class FileLogger implements AutoCloseable {
    private PrintWriter writer;
    private String filename;
    private int entriesWritten = 0;

    public FileLogger(String filename) throws IOException {
        this.filename = filename;
        this.writer = new PrintWriter(new FileWriter(filename, true));
        System.out.println("Логгер открыт: " + filename);
    }

    public void log(String level, String message) {
        String timestamp = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("yyyy-MM-dd HH:mm:ss"));
        writer.printf("[%s] [%s] %s\n", timestamp, level, message);
        writer.flush(); // Сбрасываем буфер
        entriesWritten++;
    }

    public void info(String message) {
```

```

        log("INFO", message);
    }

    public void error(String message) {
        log("ERROR", message);
    }

    public void warning(String message) {
        log("WARNING", message);
    }

    @Override
    public void close() {
        writer.close();
        System.out.println("Логгер закрыт: " + filename + " (записей: " + entriesWritten + ")");
    }

    public static void main(String[] args) {
        try (FileLogger logger = new FileLogger("app.log")) {
            logger.info("Приложение запущено");
            logger.info("Инициализация завершена");
            logger.warning("Конфигурация не найдена, используются значения по умолчанию");

            // Симулируем ошибку
            try {
                int result = 10 / 0;
            } catch (ArithmeticException e) {
                logger.error("Ошибка: " + e.getMessage());
            }

            logger.info("Работа завершена");
        } catch (IOException e) {
            System.out.println("Не удалось открыть файл лога: " + e.getMessage());
        }
        // logger.close() вызвался автоматически!

        // Прочитаем записанный файл
        System.out.println("\n--- Содержимое app.log ---");
        try (BufferedReader reader = new BufferedReader(new FileReader("app.log"))) {
            String line;
            while ((line = reader.readLine()) != null) {
                System.out.println(line);
            }
        } catch (IOException e) {
            System.out.println("Ошибка чтения: " + e.getMessage());
        }
    }
}

```

Что выводится при вызове close()?

- При автоматическом закрытии логгера срабатывает @Override метод close().
- Выводится строка о подтверждении закрытии файла и указания точного количества сделанных за сессию записей.

Почему transient здесь не применяется, а flush() нужен?

- Ключевое слово transient используется исключительно при сериализации объектов. Поле не нужно сохранять в байтовый поток при превращении объекта в файл.
- FileLogger не сериализуется. Поля writer и filename нужны для текущей работы запущенной программы, а не для сохранения состояния самого объекта логгера.

▼ result

```
Логгер открыт: app.log
Логгер закрыт: app.log (записей: 5)

--- Содержимое app.log ---
[2026-05-16 16:34:56] [INFO] Приложение запущено
[2026-05-16 16:34:56] [INFO] Инициализация завершена
[2026-05-16 16:34:56] [WARNING] Конфигурация не найдена, используются значения по умолчанию
[2026-05-16 16:34:56] [ERROR] Ошибка: / by zero
[2026-05-16 16:34:56] [INFO] Работа завершена

Process finished with exit code 0
```

Задание 4.4: Цепочка исключений

- Изучите и запустите трёхуровневую систему.

Объясните:

- (1) что выведет e.getCause().getMessage()?
- (2) для чего используется цепочка исключений?
- (3) какова разница между getMessage() и getCause().getMessage()?

1. Что выведет e.getCause().getMessage() ?

- Return сообщение корневого исключения, которое произошло на уровне базы данных.

2. Для чего используется цепочка исключений?

- **Инкапсуляция и абстракция.** Верхние слои приложения не должны знать о деталях реализации нижних слоев. Viewer слой получает ошибку бизнес-логики, а не ошибку базы данных.
- **Сохранение контекста.** Можно добавить важные высокоуровневые детали (например, id, при котором упал запрос), не теряя при этом техническую причину сбоя.
- **Упрощение отладки.** Можно увидеть в логах полный путь ошибки от места её зарождения до места перехвата.

3. Разница между getMessage() и getCause().getMessage() ?

- getMessage(): return текстовое описание текущего исключения. Формируется на уровне сервиса и объясняет, какая бизнес-операция сорвалась.
- getCause().getMessage(): return текстовое описание предыдущего исключения, которое стало триггером для текущего.

Решение

```
package practice_4.task_4_4;

class DatabaseLayer {
    public String fetchData(int id) throws Exception {
        if (id <= 0) {
            throw new Exception("SQL Error: Invalid ID " + id);
        }
        return "данные_" + id;
    }
}

class DataService {
    private DatabaseLayer db = new DatabaseLayer();

    public String getData(int id) throws ServiceException {
        try {
            return db.fetchData(id);
        } catch (Exception e) {
            throw new ServiceException("Не удалось получить данные для id=" + id, e);
        }
    }
}

class ServiceException extends Exception {
    public ServiceException(String message, Throwable cause) {
        super(message, cause);
    }
}

public class Application {
    public static void main(String[] args) {
        DataService service = new DataService();

        try {
            System.out.println(service.getData(1));
        } catch (ServiceException e) {
            System.out.println("Ошибка: " + e.getMessage());
        }

        try {
            System.out.println(service.getData(-1));
        } catch (ServiceException e) {
            System.out.println("Ошибка сервиса: " + e.getMessage());
            System.out.println("Причина: " + e.getCause().getMessage()); // Исходное исключение
            System.out.println("\nПолный стек:");
            e.printStackTrace();
        }
    }
}
```

```
}  
}
```

▼ result

```
данные_1  
Ошибка сервиса: Не удалось получить данные для id=-1  
Причина: SQL Error: Invalid ID -1  
  
Полный стек:  
practice_4.task_4_4.ServiceException Create breakpoint : Не удалось получить данные для id=-1  
    at practice_4.task_4_4.DataService.getData(Application.java:22)  
    at practice_4.task_4_4.Application.main(Application.java:48)  
Caused by: java.lang.Exception Create breakpoint : SQL Error: Invalid ID -1  
    at practice_4.task_4_4.DatabaseLayer.fetchData(Application.java:8)  
    at practice_4.task_4_4.DataService.getData(Application.java:20)  
    ... 1 more  
  
Process finished with exit code 0
```

Часть 5: Самостоятельная работа

Задание 5.1: Обобщённый кэш

- Реализуйте обобщённый LRU-кэш `SimpleCache<K, V>` с ограниченным размером.
- Используйте `LinkedHashMap` с `accessOrder=true` и переопределённым `removeEldestEntry`. Методы: `put(K key, V value)`, `get(K key)`, `containsKey(K key)`, `size()`, `clear()`.
- Продемонстрируйте: создайте кэш ёмкостью 3, добавьте `a=1`, `b=2`, `c=3`. Добавьте `d=4` — должен вытеснить наименее используемый. Проверьте, что `get("a")` вернёт `null`.

Решение

```
package practice_4.task_5_1;  
  
import java.lang.reflect.Field;  
import java.util.*;  
  
public class SimpleCache<K, V>{  
    private final LinkedHashMap<K, V> linkedHashMap;  
    public SimpleCache(int capacity){  
        this.linkedHashMap=new LinkedHashMap<K, V>(capacity, 0.75f, true){
```



```

        @Override
        protected boolean removeEldestEntry(Map.Entry<K,V> eldest){
            return size() > capacity;
        }
    };
    if (!getAccessOrderValue(this.linkedHashMap)){
        throw new IllegalArgumentException("accessOrder должен быть true");
    }
}

public static boolean getAccessOrderValue(LinkedHashMap<?,?> linkedHashMap){
    try {
        Field field = LinkedHashMap.class.getDeclaredField("accessOrder");
        field.setAccessible(true); // открывает доступ к private полю
        return (boolean) field.get(linkedHashMap);
    } catch (Exception e){
        throw new RuntimeException("Не удалось получить доступ к полю accessOrder", e);
    }
}

public void put(K key, V value){ linkedHashMap.put(key,value);}
public V getKey(K key){ return linkedHashMap.get(key);}
public boolean containsKey(K key){ return linkedHashMap.containsKey(key);}
public int size(){ return linkedHashMap.size();}
public void clear(){ linkedHashMap.clear();}

@Override
public String toString() {
    return "SimpleCache{" +
        "linkedHashMap=" + linkedHashMap +
        '}';
}
}

class Test {
    public static void main(String[] args) {
        SimpleCache<String, Integer> cache = new SimpleCache<>(3);
        cache.put("a", 1);
        cache.put("b", 2);
        cache.put("c", 3);
        System.out.println("Кэш после добавления 3 элементов: " + cache);
        cache.put("d", 4);
        System.out.println("Кэш после добавления 4 элемента: " + cache);
        System.out.println("Получение элемента 'a' (ожидается null): " + cache.getKey("a"));
        System.out.println("Есть ли такой ключ 'b': " + cache.containsKey("b"));
        System.out.println("Размер: " + cache.size());
    }
}

```

▼ result

```
Кэш после добавления 3 элементов: SimpleCache{linkedHashMap={a=1, b=2, c=3}}
Кэш после добавления 4 элемента: SimpleCache{linkedHashMap={b=2, c=3, d=4}}
Получение элемента 'a' (ожидается null): null
Есть ли такой ключ 'b': true
Размер: 3

Process finished with exit code 0
```

Задание 5.2: Обобщённый Result тип

Реализуйте обобщённый тип Result — обёртку для результата операции:

- Фабричные методы: static Result success(T value) и static Result failure(Exception error).
- * isSuccess(), getValue(), getError().
- * getOrDefault(T defaultValue) – возвращает значение при успехе, иначе defaultValue.
- * <R> Result<R> map(Function<T, R> mapper) – трансформирует значение при успехе; при неуспехе возвращает failure(error).
- * toString().

Реализуйте static Result divide(int a, int b), продемонстрируйте divide(10,2) и divide(10,0), getOrDefault, цепочку map.

Решение

```
package practice_4.task_5_2;

import java.util.function.Function;

public class Result<T> {
    private final T value;
    private final Exception error;
    private Result(T value){
        this.value=value;
        this.error=null;
    }
    private Result(Exception error){
        this.value=null;
        this.error=error;
    }
    public static <T> Result<T> success(T value){ // фабричный метод для успешного результата
        return new Result<>(value);
    }
}
```

```

public static <T> Result<T> failure(Exception error){ // фабричный метод для ошибки
    return new Result<>(error);
}
public boolean isSuccess(){ // проверка на успешность операции
    return error == null;
}
public T getValue(){ // получение значения
    return value;
}
public Exception getError(){ // получение ошибки
    return error;
}
public T getOrDefault(T defaultValue){ // при успехе получаем значение, при ... дефолтное
    return isSuccess() ? value:defaultValue;
}
public <R> Result<R> map(Function<T, R> mapper){
    if (!isSuccess()){
        return Result.failure(this.error);
    }
    try {
        return Result.success(mapper.apply(this.value));
    } catch (Exception e){
        return Result.failure(e);
    }
}
@Override
public String toString(){
    if (isSuccess()){
        return "Result.success={value=" + value + "}";
    } else{
        return "Result.failure={error=" + error.getClass().getSimpleName() + "}";
    }
}
}

class Test {
    public static Result<Integer> divide(int a, int b) {
        if (b == 0) {
            return Result.failure(new ArithmeticException("Деление на ноль невозможно"));
        }
        return Result.success(a / b);
    }
}

public static void main(String[] args) {
    System.out.println("1. Проверка divide(10, 2)");
    Result<Integer> successResult = divide(10, 2);
    System.out.println("Результат операции: " + successResult);
    System.out.println("isSuccess(): " + successResult.isSuccess());
    System.out.println("getValue(): " + successResult.getValue());
    System.out.println("getOrDefault(100): " + successResult.getOrDefault(100));

    System.out.println("\n2. Проверка divide(10, 0)");
    Result<Integer> failureResult = divide(10, 0);
    System.out.println("Результат операции: " + failureResult);
    System.out.println("isSuccess(): " + failureResult.isSuccess());
    System.out.println("getError(): " + failureResult.getError());
}

```

```

        System.out.println("getOrDefault(100): " + failureResult.getOrDefault(100));

        System.out.println("\n3. Проверка цепочки map (Успех)");
        Result<Integer> chainSuccess = divide(10, 2).map(res -> "Результат: " + res).map(String::length)
        System.out.println("Итог цепочки map для успеха: " + chainSuccess);

        System.out.println("\n4. Проверка цепочки map (Ошибка)");
        Result<Integer> chainFailure = divide(10, 0).map(res -> "Результат: " + res).map(String::length)
        System.out.println("Итог цепочки map для ошибки: " + chainFailure);
    }
}

```

▼ result

```

1. Проверка divide(10, 2)
Результат операции: Result.success={value=5}
isSuccess(): true
getValue(): 5
getOrDefault(100): 5

2. Проверка divide(10, 0)
Результат операции: Result.failure={error=ArithmeticException}
isSuccess(): false
getError(): java.lang.ArithmeticException: Деление на ноль невозможно
getOrDefault(100): 100

3. Проверка цепочки map (Успех)
Итог цепочки map для успеха: Result.success={value=12}

4. Проверка цепочки map (Ошибка)
Итог цепочки map для ошибки: Result.failure={error=ArithmeticException}

Process finished with exit code 0

```

Контрольные вопросы

1. В чём разница между нестатическим внутренним классом и статическим вложенным классом?
 - Нестатический имеет доступ ко всем полям внешнего класса, статический только к статическим.
2. Почему для создания экземпляра нестатического внутреннего класса снаружи нужен экземпляр внешнего класса?

- Нестатический внутренний класс неявно хранит ссылку на экземпляр внешнего класса, который его создал.
- Дает прямой доступ к приватным полям и методам этого экземпляра (без родителя не может существовать).

3. Что такое стирание типов (type erasure)? Приведите пример того, что нельзя сделать из-за этого.

- Параметры типа существуют только во время компиляции. В байт-коде заменяются на Object (в случае), а если extends по какому-то классу, например, Number, то и замена на него.

Что нельзя делать:

```
public class Container<T>{
    T obj = new T();// НЕЛЬЗЯ создать экземпляр T
    T[] array = new T[10];// НЕЛЬЗЯ создавать массивы типа T
    if (obj instanceof T){}// НЕЛЬЗЯ использовать instanceof с T
    Class<T> c = T.class// НЕЛЬЗЯ получить класс T
}
```

Что можно делать:

```
public class Container<T>{
    T value; // МОЖНО объявлять переменные типа T
    List<T> list = new ArrayList<>(); // МОЖНО создавать коллекции
}
```

Демонстрация стирания типов во время исполнения:

```
class TypeErasureDemo<T> {
    public void check(Object obj) {
        // if (obj instanceof T) {} – Ошибка: нельзя использовать instanceof с параметром типа
        System.out.println("Тип T стёрт до Object на этапе исполнения");
    }
}

// Использование:
TypeErasureDemo<String> ted = new TypeErasureDemo<>();
ted.check("Test");
```

4. Объясните принцип PECS. Когда использовать ? extends T и когда ? super T?

- PECS = wildcard, когда не знаем какого типа передаём структуры данных в дженериках. Такое правило, чтобы не возникало ошибок.
- extends T = для чтения.
- super T = для записи.

Копирование коллекции:

```
public static <T> void copy(List<? super T> dest, List<? extends T> src) {  
    for (T item : src) {    // src – producer, читаем из него  
        dest.add(item);    // dest – consumer, пишем в него  
    }  
}
```

5. Чем отличается throws от throw?

- throw = выбрасывает исключение. throws = обрабатывает случай, когда исключение может появиться (checked).

6. В каких случаях нужно создавать собственные checked исключения, а в каких — unchecked?

- Классического использования исключений мало. Иногда для конкретной бизнес-задачи, нужно создавать свои исключения.

7. Что произойдёт, если исключение возникнет внутри блока finally?

- Полное перекрытие предыдущего исключения, которое могло возникнуть в блоках.

8. Какой интерфейс должен реализовывать класс для использования в try-with-resources?

- Класс ресурса должен реализовывать интерфейс AutoCloseable (наследник Closeable).

9. Для чего используется цепочка исключений (exception chaining)?

- Польза в сохранении оригинальное исключение при выбрасывании нового.
- Не терять информацию о первой ошибки/причина возникновения при перемещении между уровнями кода.

10. Можно ли создать массив обобщённого типа (new T[10])? Объясните почему.

- Нет, из-за механизма стирания типов (Type Erasure) в Java.
- При работе программы информация о типе T исчезает. JVM не знает: массив - тип - размер в байтах, который нужно выделить в памяти.
- Массивы в Java должны жестко знать свой тип во время выполнения, а дженерики существуют только на этапе компиляции.